

EGMCC: A High-Performance Multi-Core CPU Engine for Large-Scale Network Analysis

Su-Su Zhang¹, Yuzhe Wang¹, Min Gao², Qingyuan Gong¹, Xin Wang¹, Yang Chen^{1,*}

¹College of Computer Science and Artificial Intelligence, Fudan University, China

²School of Information Science and Engineering, East China University of Science and Technology, China

{zhangss25, yuzhewang25}@m.fudan.edu.cn, {gongqingyuan, xinw, chenyang}@fudan.edu.cn, mgao97@ecust.edu.cn

Abstract—Large-scale networks are increasingly prevalent across social, biological, and information systems. However, many foundational network analysis algorithms remain computationally expensive on commodity hardware. This performance gap limits the practical application of standard network measures to contemporary complex systems. We present EGMCC (Easy-Graph Multi-Core CPU), a high-performance multicore backend to accelerate three families of network analysis algorithms: centrality measures, structural hole metrics, and path-based routines. Implemented in C++, EGMCC combines OpenMP-based shared-memory parallelism, compact graph representations, workload-aware scheduling, and algorithm-specific optimizations, all while retaining EasyGraph’s accessible Python interface. Extensive experiments on diverse real-world networks demonstrate average speedups of $56\times$ and $737\times$ over sequential libraries like *igraph* and *NetworkX*, respectively, alongside substantial performance advantages over state-of-the-art parallel baselines such as *graph-tool* and *NetworkKit*. This enables significantly larger networks to be analyzed without specialized GPU hardware. The accompanying demonstration allows users to select datasets and algorithms, configure multicore execution, and directly compare the performance and outputs of different execution backends. These results illustrate that systematic multicore optimization can substantially enhance the efficiency and accessibility of foundational network analysis.

Index Terms—Multi-Core CPU, Open Multi-Processing, Graph Algorithms, System Optimization

I. INTRODUCTION

Complex networks are widely used to model interconnected systems in social computing, biology, transportation, communication, and other data-intensive domains [1]. Their analysis relies on foundational algorithms such as centrality measures [2], structural-hole (SH) metrics [3], community detection [4], and path computation [5]. These algorithms support tasks ranging from influential-node identification to brokerage analysis and connectivity assessment. However, as real-world networks continue to grow, the computational cost of these basic routines increasingly limits large-scale analysis, repeated experimentation, and interactive exploration.

Although modern commodity hardware provides abundant multicore resources, existing network analysis tools fail to fully exploit this potential. Shared-memory parallelism offers an accessible strategy to accelerate large-scale analysis without requiring GPUs or distributed infrastructure, yet widely used libraries like *igraph* and *NetworkX* rely heavily on sequential implementations. Furthermore, while hybrid frameworks like *EasyGraph* distribute computations at the Python level, this

coarse-grained orchestration cannot parallelize the internal execution of serial C++ kernels [6]. It also lacks control over low-level memory layout, fine-grained task partitioning, and synchronization. Migrating parallelism directly into the native backend eliminates Python-side communication overhead, avoids repeated boundary transitions, and allows threads to access shared graph structures efficiently. Consequently, systematically redesigning the core execution engine with fine-grained native parallelism is essential to overcome current performance bottlenecks and scale network analysis on commodity hardware.

To address these limitations, we present EGMCC, a high-performance multicore extension of the *EasyGraph* C++ backend. EGMCC provides a unified native execution architecture for three families of network algorithms: centrality measures, structural-hole metrics, and path-based routines. By shifting parallelism from Python to native OpenMP-based C++ kernels, EGMCC enables fine-grained control over data access and synchronization. Efficiently parallelizing these algorithms is inherently challenging due to highly irregular graph workloads, which often suffer from severe load imbalances, poor cache locality, and synchronization overheads. EGMCC addresses these challenges through three complementary optimizations. First, it utilizes compact adjacency representations, such as compressed sparse row (CSR) formats, to improve memory locality and reduce graph-access overhead [7]. Second, it employs workload-aware OpenMP scheduling to evenly distribute irregular computations across threads [8]. Third, it adopts thread-local states and intermediate-result reuse to minimize data races and redundant calculations.

Crucially, these optimizations are adapted to the specific computational characteristics of each algorithm family rather than applied as uniform loop transformations. Designed as an integrated system, EGMCC allows users to invoke these accelerated routines through familiar Python APIs with minimal changes to their workflows, effectively translating multicore CPU potential into practical, accessible acceleration.

The main contributions of this work are as follows:

- We develop EGMCC, a high-performance multicore C++ backend that accelerates three families of foundational network algorithms while preserving a user-friendly Python interface.
- We integrate system-level and algorithm-specific optimizations, such as CSR storage, workload-aware schedul-

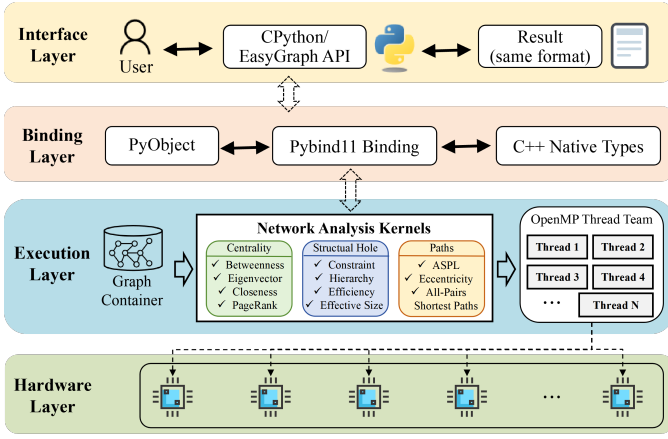


Fig. 1. The layered execution framework of EGMCC.

ing, and thread-local computation, to efficiently overcome irregular workloads and memory bottlenecks.

- We comprehensively evaluate EGMCC on multiple real-world datasets and demonstrate its massive runtime speedups against state-of-the-art network analysis libraries.

II. SYSTEM DESIGN AND OPTIMIZATION

A. System Overview

As shown in Figure 1, EGMCC follows a layered architecture that separates the user-facing Python interface from the performance-critical native execution backend. Users invoke network analysis routines through the existing EasyGraph APIs, while an internal dispatcher routes supported operations to the optimized C++ kernels. Before execution, the input graph is converted into compact native data structures that support efficient parallel traversal. The multicore execution engine then coordinates OpenMP threads, workload assignment, intermediate states, and result aggregation according to the computational characteristics of each algorithm family.

B. Graph Data Processing

EGMCC combines multicore execution with data-layout and algorithm-engineering optimizations. Rather than enforcing one graph format on all kernels, the backend selects a representation suited to each access pattern. Traversal-oriented algorithms use a compact linked adjacency representation, iterative centrality algorithms use transposed CSR structures for contiguous incoming-neighbor access, MST algorithm operates on compact COO-style edge arrays, and structural-hole measures directly access native C++ adjacency structures. Reusable graph representations and metadata are retained where applicable to reduce repeated Python-to-C++ conversion and graph-construction overhead.

C. Thread-Private Workspace

To leverage the relatively abundant memory capacity of CPU-based platforms and avoid data races during parallel execution, EGMCC allocates a thread-private workspace for

each worker thread where applicable. Each workspace maintains the mutable intermediate states required by its assigned tasks, including distance arrays, traversal queues, stacks, path counters, dependency values, and local accumulators. Threads can therefore perform most traversal operations independently without repeatedly accessing or modifying shared intermediate data. Read-only graph representations are shared across threads, whereas mutable traversal states remain private to individual workers. Partial results are locally accumulated and subsequently combined through deferred aggregation or OpenMP reduction when required. Moreover, the allocated buffers are reused across multiple tasks assigned to the same thread, reducing repeated memory allocation and initialization overhead. This design deliberately trades additional memory consumption for lower synchronization and contention costs, making it particularly effective for traversal-intensive centrality algorithms such as betweenness and closeness centrality.

D. Design of Multicore Execution

Network analysis algorithms exhibit substantially different dependency structures and workload distributions. Therefore, EGMCC applies different loop-level parallelization strategies for different algorithms. Specifically, it selects the parallel decomposition according to the computational structure of each algorithm.

Traversal-level parallelism. Betweenness centrality, closeness centrality, multi-source Dijkstra, and average shortest path length require repeated graph traversals from different source vertices. EGMCC treats each single-source BFS or Dijkstra execution as an independent task and distributes these traversal tasks across OpenMP threads. Dynamic scheduling is adopted because traversals originating from different sources may exhibit substantially different computational costs due to variations in reachable subgraphs, traversal depth, and dependency accumulation. This decomposition exposes coarse-grained parallelism while reducing workload imbalance among threads.

Node-level parallelism. Structural-hole measures and iterative centrality algorithms are decomposed into per-node or per-row update tasks rather than complete graph traversals. Constraint, Effective Size, Hierarchy, and Efficiency independently compute a local structural value for each target node after shared normalization quantities have been prepared. Eigenvector and Katz centrality update each node by processing one row of a transposed CSR representation, while PageRank distributes contribution propagation across vertices. Depending on the write dependencies of individual kernels, EGMCC uses disjoint output regions, OpenMP reductions, thread-local intermediate states, or atomic accumulation to preserve correctness.

Edge-level parallelism. For minimum spanning tree computation, EGMCC parallelizes the edge-scanning phase of the Borůvka algorithm. During each iteration, threads inspect the current active edges and identify candidate minimum outgoing edges for individual connected components. Atomic compare-and-swap operations resolve concurrent updates to the same

component candidate. Component merging is subsequently performed using a union-find structure, after which intra-component edges are removed from the active edge set to reduce the amount of work in later iterations.

E. Specialized Execution Paths

EGMCC provides specialized execution paths according to the structural and numerical properties of the input graph. For shortest-path-based algorithms, unweighted networks are processed using breadth-first search, whereas weighted networks invoke Dijkstra's algorithm. This avoids the priority-queue operations and floating-point distance updates required by weighted shortest-path computation when all edges have equal cost. Similar specialization is applied to iterative centrality algorithms. Eigenvector centrality and PageRank inspect whether the input network contains non-unit edge weights and bypass unnecessary weight accesses and multiplications when the graph is effectively unweighted. By selecting a lightweight execution path whenever possible, EGMCC reduces arithmetic operations, memory accesses, and auxiliary data-structure overhead for common unweighted-network workloads.

F. Computation Reuse and Kernel Fusion

EGMCC further reduces runtime by eliminating repeated initializations and memory movements within individual kernels. Traversal-intensive algorithms employ timestamp-based state management, avoiding the need to clear complete vertex arrays before every BFS or Dijkstra execution. Additionally, shared intermediate quantities are computed once and reused. For instance, structural-hole kernels precompute node strengths before evaluating targets, while the Borůvka minimum spanning tree implementation progressively removes merged edges to shrink the active search space in subsequent iterations.

For iterative centrality algorithms, sparse aggregation, score updating, and convergence checking are fused into minimal memory passes. Consecutive iteration vectors are also exchanged through buffer swapping rather than explicit copying. Ultimately, these kernel-level techniques drastically reduce redundant computation and unnecessary memory traffic, directly complementing the overarching performance gains obtained from multicore parallelism.

G. Python Integration and Usability

Although EGMCC is implemented in C++, users interact with the system entirely through the existing EasyGraph Python interface. Pybind11 serves as the binding layer between the Python frontend and the native backend. Python graph objects and algorithm parameters are passed to the corresponding C++ kernels, and the computed results are subsequently converted into Python-compatible objects and returned to the calling program. This integration hides low-level details such as thread creation, native memory management, graph representation selection, and OpenMP scheduling from scientific users. Consequently, EGMCC combines the execution efficiency of compiled C++ kernels with the accessibility and flexibility of Python-based network analysis workflows.

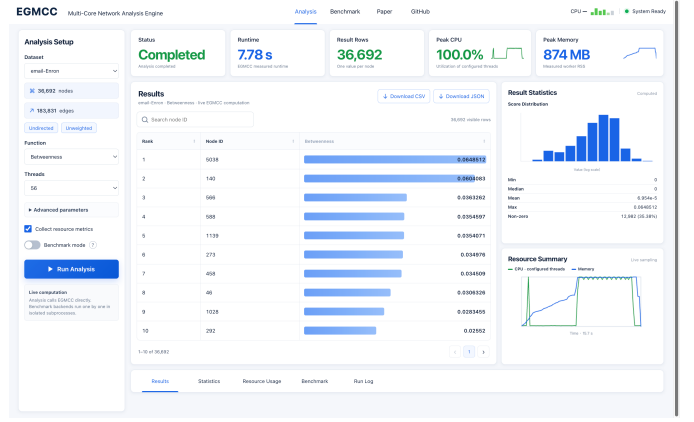


Fig. 2. Dashboard of the EGMCC.

III. DEMONSTRATION

A. Demo Setup

All experiments are conducted on a Linux server equipped with an Intel Xeon E5-2683 v3 CPU @ 2.00GHz and 256 GB of RAM. To ensure a fair comparison, all baseline libraries are evaluated using their specific versions: NetworkX 3.6.1, igraph 1.0.0, graph-tool 3.0 and NetworKit 11.2.1. Furthermore, to ensure the reliability of the results, all reported execution times are the average of 30 independent runs.

The source code and a demonstration of EGMCC are available at <https://github.com/Zhang-S-S/EGMCC>.

B. Evaluation Datasets

To validate the performance of EGMCC, we evaluate it on four real-world datasets from different domains:

- soc-sign-bitcoin-otc (OTC) [9]: A Bitcoin web of trust network with 5,881 nodes and 35,592 edges, where directed and signed edges represent trust ratings (-10 to +10) between anonymous traders.
- email-Enron (Enron) [10]: An email communication network with 36,692 nodes and 183,831 edges, where undirected edges denote email exchanges between addresses.
- musae-github (GitHub) [11]: A developer social network with 37,700 nodes and 289,003 edges, where undirected edges represent mutual follower relationships.
- web-NotreDame (NotreDame) [12]: A directed graph with 325,729 nodes and 1,497,134 edges, where edges represent hyperlinks between university domain pages.

C. Case Study

We present a case study computing Betweenness centrality on the Enron dataset, which is shown in Figure 2. The system interface allows users to configure their analytical workflows by selecting specific networks, defining target algorithms, and explicitly allocating the number of worker threads. In this demonstration, setting the environment to 56 threads directly engages the underlying multicore execution engine, translating user-defined parameters into optimized parallel computations without requiring low-level programming.

TABLE I
EXECUTION TIME COMPARISON OF DIFFERENT LIBRARIES (S). “—”
INDICATES THAT THE CORRESPONDING FUNCTION IS UNAVAILABLE.

Category	Function	Library	OTC	Enron	GitHub	NotreDame
Centrality	Betweenness	EGMCC	0.205	7.164	18.781	47.024
		graph-tool	0.288	14.305	32.178	237.309
		NetworkKit	0.425	24.410	53.417	410.674
		igraph	8.600	144.877	341.384	886.386
		NetworkX	310.628	9414.009	17203.740	137421.694
Path	Eccentricity	EGMCC	0.034	3.694	8.848	2.713
		graph-tool	—	—	—	—
		NetworkKit	1.148	57.011	108.938	95.563
		igraph	1.380	66.706	136.898	104.308
		NetworkX	16.101	948.427	1769.247	1771.817
SH	Constraint	EGMCC	0.035	0.344	1.365	2.370
		graph-tool	—	—	—	—
		NetworkKit	—	—	—	—
		igraph	0.127	2.604	3.746	13.286
		NetworkX	0.331	4.079	33.257	60.487

Following the execution, EGMCC delivers immediate analytical results alongside comprehensive hardware profiling. The primary dashboard highlights a highly efficient runtime of 7.78s while tracking hardware utilization, which reveals a peak CPU usage of 100% and a peak memory consumption of 874 megabytes. The computed metrics are structured into a searchable and rank-ordered result table spanning all evaluated vertices, complete with inline visual bars to facilitate the rapid identification of influential nodes.

Additionally, EGMCC shows a score distribution histogram and dynamic resource tracking charts, enabling researchers to simultaneously evaluate the algorithmic outputs and monitor the hardware efficiency of the multicore execution. The statistical panel reveals that 12,982 nodes possess non-zero betweenness values. This highly skewed distribution is typical for scale-free graphs and is visually reinforced by the logarithmic histogram. Concurrently, the live resource sampling chart traces the memory allocation and thread activity over the execution lifecycle, proving that the underlying engine maintains stable hardware utilization while delivering rapid analytical feedback.

D. Baseline

To evaluate the performance of EGMCC, we compare it with three widely adopted network analysis libraries: (1) NetworkX [13], the most widely used network analysis library; (2) igraph [14], a highly optimized library with a C++ core, selected as a rigorous baseline for execution efficiency; (3) graph-tool [15] and NetworkKit [16], both of which are high-performance libraries that natively support parallel computing, serving as direct comparative baselines for multicore scalability. By benchmarking against state-of-the-art libraries, we demonstrate that our specific multi-core threading strategies and memory layout optimizations provide superior acceleration without sacrificing the user-friendly Python interface.

E. Benchmark Results

Beyond the case study, we further benchmark the runtime performance of EGMCC against baseline libraries across diverse datasets, with representative functions summarized in Table I. Overall, EGMCC consistently outperforms all baselines.

For centrality algorithms, EGMCC computes Betweenness nearly $9\times$ faster than the parallel baseline NetworkKit on the large-scale NotreDame dataset, and achieves up to a $268\times$ speedup over igraph for Closeness, while iterative algorithms like PageRank and Eigenvector exhibit similar massive performance gains.

For path computation, EGMCC calculates Eccentricity on NotreDame up to $35\times$ faster than NetworkKit and $653\times$ faster than NetworkX, while metrics like All-Pairs Shortest Paths operate significantly faster than graph-tool and igraph. Finally, EGMCC provides comprehensive coverage and superior speed for structural hole metrics, which are notably absent in parallel baselines like NetworkKit. It computes Constraint $5.6\times$ faster than igraph on NotreDame and evaluates other complex metrics in under two seconds, effectively avoiding the out-of-memory errors frequently encountered by NetworkX.

REFERENCES

- [1] S. H. Strogatz, “Exploring complex networks,” *Nature*, vol. 410, no. 6825, pp. 268–276, 2001.
- [2] M. Wang, H. Wang, and H. Zheng, “A mini review of node centrality metrics in biological networks,” *International Journal of Network Dynamics and Intelligence*, vol. 1, no. 1, pp. 99–110, 2022.
- [3] Z. Zhao, D. Li, Y. Sun, R. Zhang, and J. Liu, “Ranking influential spreaders based on both node k-shell and structural hole,” *Knowledge-Based Systems*, vol. 260, p. 110163, 2023.
- [4] X. Su, S. Xue, F. Liu, J. Wu, J. Yang, C. Zhou, W. Hu, C. Paris, S. Nepal, D. Jin, Q. Z. Sheng, and P. S. Yu, “A comprehensive survey on community detection with deep learning,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 4, pp. 4682–4702, 2024.
- [5] E.-R. Ardelean, R. L. Portase, R. Potolea, and M. Dînsoreanu, “A path-based distance computation for non-convexity with applications in clustering,” *Knowledge and Information Systems*, vol. 67, no. 2, pp. 1415–1453, 2025.
- [6] M. Gao, Z. Li, R. Li, C. Cui, X. Chen, B. Ye, Y. Li, W. Gu, Q. Gong, X. Wang, and Y. Chen, “EasyGraph: A multifunctional, cross-platform, and effective library for interdisciplinary network analysis,” *Patterns*, vol. 4, no. 10, p. 100839, 2023.
- [7] B. Wheatman and H. Xu, “Packed compressed sparse row: A dynamic graph representation,” in *2018 IEEE High Performance Extreme Computing Conference (HPEC)*, 2018, pp. 1–7.
- [8] L. Dagum and R. Menon, “OpenMP: An industry-standard API for shared-memory programming,” *IEEE Computational Science and Engineering*, vol. 5, no. 1, pp. 46–55, 1998.
- [9] S. Kumar, F. Spezzano, V. S. Subrahmanian, and C. Faloutsos, “Edge weight prediction in weighted signed networks,” in *2016 IEEE 16th International Conference on Data Mining (ICDM)*, 2016, pp. 221–230.
- [10] J. Leskovec, K. J. Lang, A. Dasgupta, and M. W. Mahoney, “Community structure in large networks: Natural cluster sizes and the absence of large well-defined clusters,” *Internet Mathematics*, vol. 6, no. 1, pp. 29–123, 2009.
- [11] B. Rozemberczki, C. Allen, and R. Sarkar, “Multi-Scale attributed node embedding,” *Journal of Complex Networks*, vol. 9, no. 2, p. cnab014, 2021.
- [12] R. Albert, H. Jeong, and A.-L. Barabási, “Internet: Diameter of the World-Wide Web,” *Nature*, vol. 401, no. 6749, pp. 130–131, 1999.
- [13] A. Hagberg, D. A. Schult, and P. J. Swart, “Exploring network structure, dynamics, and function using NetworkX,” in *Proceedings of the 7th Python in Science Conference*, 2008, pp. 11–15.
- [14] G. Csárdi and T. Nepusz, “The igraph software package for complex network research,” *InterJournal, Complex Systems*, p. 1695, 2006.
- [15] T. P. Peixoto, *Descriptive vs. Inferential Community Detection in Networks: Pitfalls, Myths and Half-Truths*. Cambridge University Press, 2023.
- [16] C. L. Staudt, A. Sazonovs, and H. Meyerhenke, “Networkit: A tool suite for large-scale complex network analysis,” *Network Science*, vol. 4, no. 4, pp. 508–530, 2016.